

PROJECT REPORT



MEDISPHERE 3D

Smart Hospital Management System

Created By:

Soubhagya Kumar Behura

Registration Number:

25BAI10757

Course:

Programming in Java

Program:

B.Tech

College:

VIT Bhopal University

Date of Submission:

18 September 2026

1. Introduction

MediSphere 3D is a smart hospital management system developed as a project for the Programming in Java course. The main idea behind the project is to bring several common hospital management activities into one organized system instead of handling each activity separately.

The system provides modules for patients, doctors, appointments, emergency cases, hospital operations, and analytics. A modern 3D-inspired interface has been used to make the application easier to explore and visually engaging.

Along with the web interface, the project contains a separate Java implementation. The Java part represents hospital entities such as patients, doctors, appointments, and emergency cases and applies core Java concepts to these real-world examples.

The project therefore combines a practical hospital-management problem with concepts learned in the Programming in Java course.

2. Problem Statement

Hospitals deal with a large amount of information related to patients, doctors, appointments, emergencies, and daily operations. When these activities are managed through separate systems or manually, it can become difficult to find information quickly and coordinate different activities.

For example, information about a doctor may be required while creating an appointment, while emergency cases may require quick access to available doctors and their specializations.

The project aims to address this problem by creating a centralized hospital-management system where different hospital activities can be organized into separate but connected modules.

The Java implementation further demonstrates how programming concepts can be applied to represent and process hospital-related information.

3. Objectives

The main objectives of MediSphere 3D are:

- To develop a centralized hospital management system.
 - To organize patient information.
 - To maintain information about doctors from different specializations.
 - To provide appointment and consultation management.
 - To organize emergency cases.
 - To display available emergency doctors.
 - To provide hospital operational information.
 - To present hospital-related statistics and analytics.
 - To apply Java programming concepts to a practical problem.
 - To demonstrate object-oriented programming using hospital-related classes.
 - To implement exception handling for invalid operations.
 - To use collections for managing multiple hospital records.
 - To create a simple and interactive user interface.
-

4. Functional Requirements

The system consists of several major functional modules.

4.1 Patient Management

The Patient Management module allows hospital information related to patients to be organized.

Main functions include:

- Patient directory
- Patient information
- Patient profiles
- Patient search
- Patient registration
- Appointment-related information

4.2 Doctor Management

The Doctor Management module provides information about doctors and their specializations.

It includes:

- Doctor directory
- Doctor profiles
- Doctor specialization
- Qualifications
- Experience
- Availability
- Consultation information

The system contains multiple specialist categories including cardiology, dentistry, neurology, orthopedics, and other medical areas.

4.3 Appointment Management

The appointment module manages consultation-related information.

It allows the system to work with:

- Patient selection
- Doctor selection
- Appointment date
- Appointment time
- Consultation type
- Appointment status

The Java implementation also performs validation when appointment-related operations are carried out.

4.4 Emergency Management

Emergency Command is divided into five sections:

- Emergency Dashboard
- Priority Queue
- Active Cases
- Available Emergency Doctors
- Emergency History

This structure allows emergency information to be organized according to the current state of a case.

4.5 Hospital Operations

This module provides information about hospital operations, including:

- Bed availability
- ICU status
- Operation theatre information
- Laboratory status
- Pharmacy information
- Staff availability

4.6 Analytics

The Analytics module presents summarized information related to:

- Patients
 - Appointments
 - Doctors
 - Departments
 - Emergency cases
-

5. Non-Functional Requirements

5.1 Usability

The interface should be simple enough for users to navigate between different hospital modules without unnecessary complexity.

5.2 Performance

The application should provide quick interaction and avoid unnecessary delays while navigating between modules.

5.3 Reliability

Invalid operations should be handled appropriately rather than causing the complete application to fail.

5.4 Maintainability

The Java implementation is separated into packages and classes so that individual components can be modified without changing the complete system.

5.5 Error Handling

The Java implementation uses exception handling and a user-defined `AppointmentException` to handle invalid appointment-related operations.

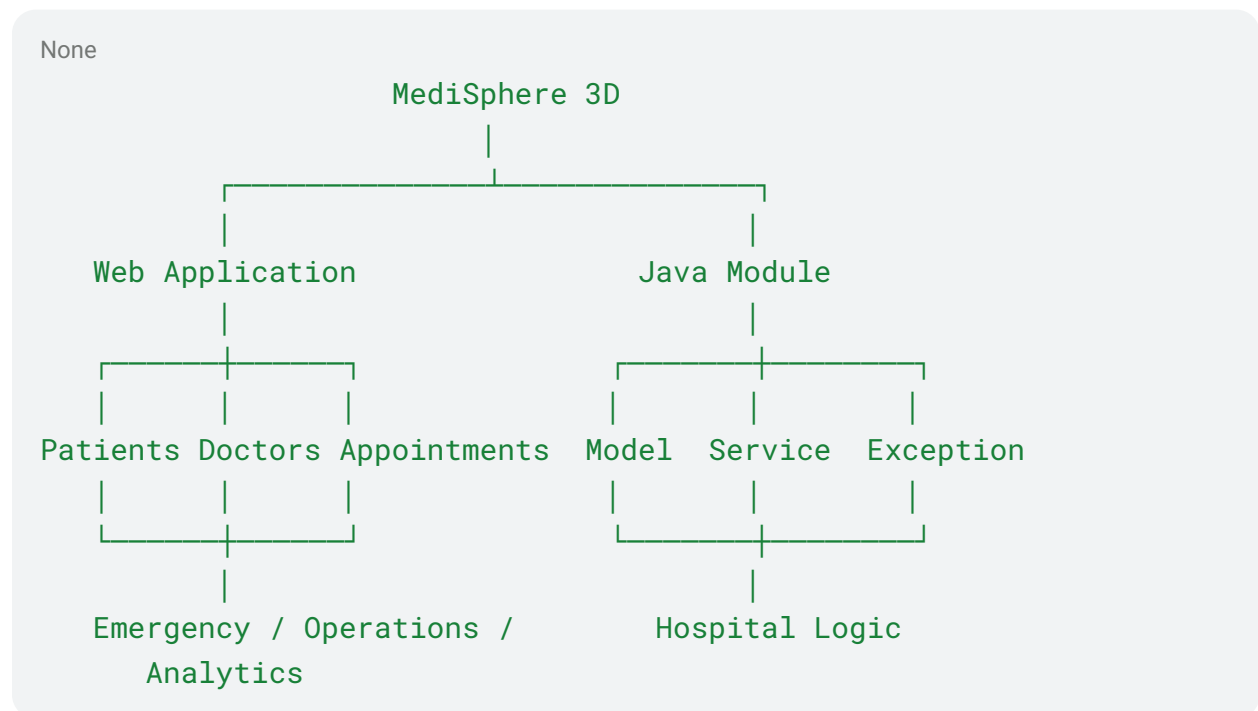
5.6 Modularity

Different hospital entities and services are represented through separate Java classes and packages.

6. System Architecture

MediSphere 3D follows a modular architecture consisting of a web application and a Java implementation.

The overall structure can be represented as:



The Java module is organized using packages such as:

None

`model`

`service`

`exception`

The `model` package contains hospital-related entities, the `service` package contains hospital operations, and the `exception` package contains custom exception handling.

7. Process Workflow

The basic workflow of the system is:

None

`Start`



`Landing Page`



`Explore System`



`Hospital Dashboard`



`Select Required Module`



<code>Patient</code>	<code>Doctor</code>	<code>Appointment</code>
<code>Management</code>	<code>Management</code>	<code>Management</code>



`Emergency / Operations / Analytics`



`View or Process Information`



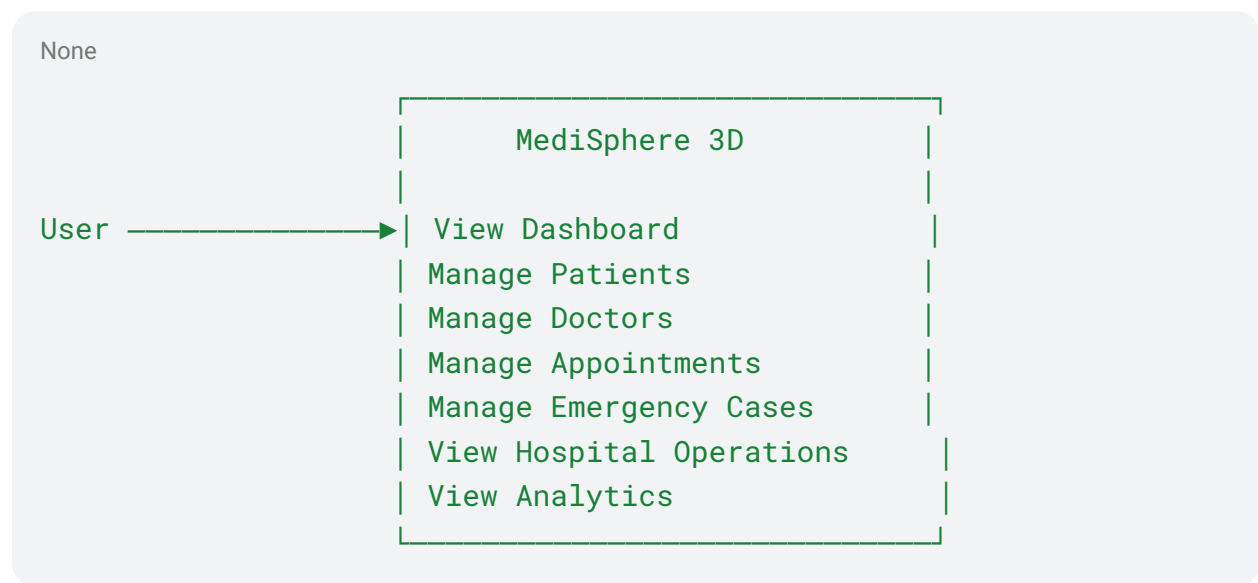
`End`

For an appointment-related operation, the Java logic validates the required information before completing the operation.

8. Design Diagrams

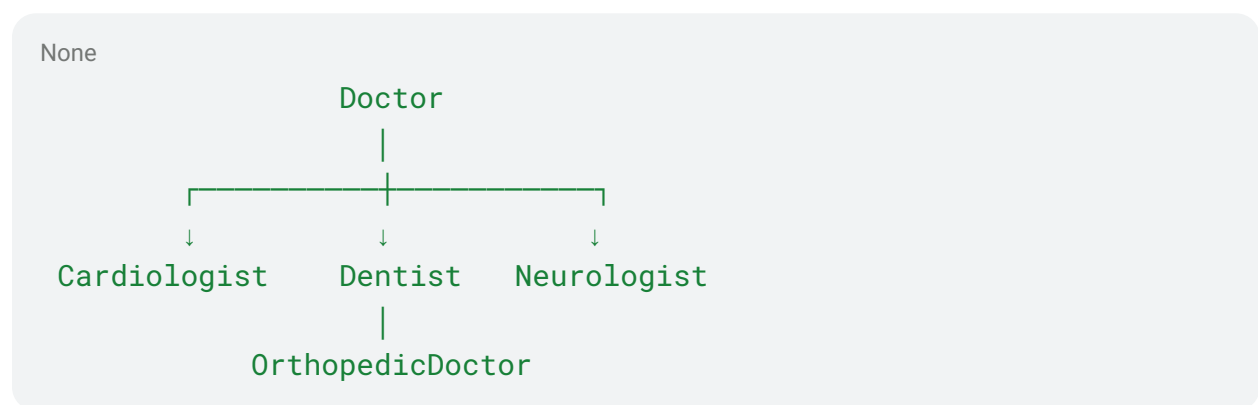
8.1 Use Case Diagram

The main actor can interact with the hospital management system through:



8.2 Class Diagram

The Java implementation follows relationships such as:

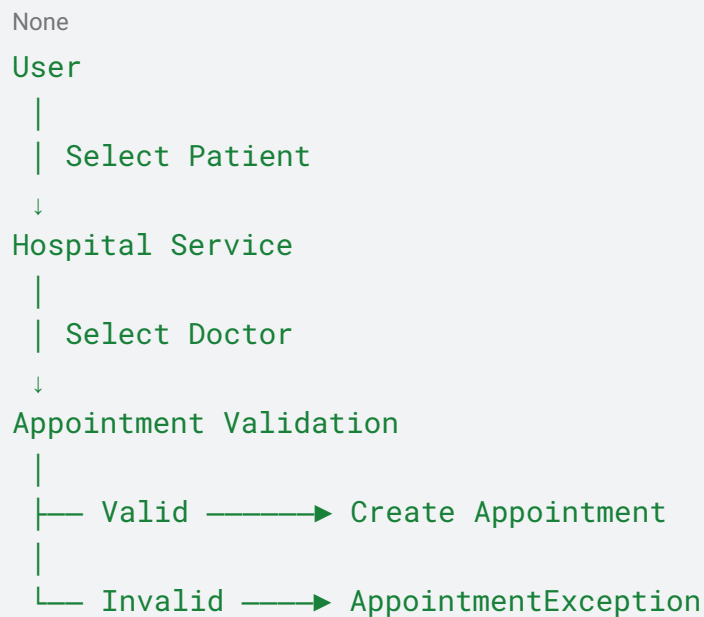




The specialist doctor classes inherit common properties and behavior from `Doctor`.

8.3 Sequence Flow

A simplified appointment flow is:



9. Java Concepts Implemented

The Java implementation in the project currently demonstrates the following concepts:

Java Concept	Implementation
Classes & Objects	Patient, Doctor, Appointment, EmergencyCase and specialist classes
Variables & Data Types	Fields using types such as <code>String</code> , <code>int</code> , <code>double</code> , and <code>boolean</code>
Methods	Hospital operations, getters, setters and consultation methods
Constructors	Constructors used to create hospital-related objects
<code>this</code> Keyword	Used within class implementation
Encapsulation	Private fields with controlled access through methods
Inheritance	Specialist doctor classes extend <code>Doctor</code>
Method Overriding	Specialist classes override doctor behavior
Polymorphism	Doctor references can represent specialist doctor objects
<code>super</code> Keyword	Used by specialist classes to call the parent constructor
Exception Handling	Java exception handling for invalid operations
<code>try-catch</code>	Used to handle exceptions
<code>throw</code>	Used when validation detects an invalid operation
<code>throws</code>	Used by methods that can produce exceptions
User-defined Exception	<code>AppointmentException</code>
User-defined Packages	<code>model</code> , <code>service</code> , and <code>exception</code>
Collections Framework	Used for managing multiple records
List / ArrayList	Hospital records are stored using <code>ArrayList</code>
<code>if-else</code>	Used for conditions and validation
<code>for</code> Loop	Used for processing records
<code>for-each</code> Loop	Used for iterating through collections

These are the concepts that are actually represented in the current Java source code.

10. Implementation Details

The Java source code is organized into separate packages to keep the implementation modular.

10.1 Model Package

The model package contains classes representing hospital entities.

Examples include:

- `Patient.java`
- `Doctor.java`
- `Cardiologist.java`
- `Dentist.java`
- `Neurologist.java`
- `OrthopedicDoctor.java`
- `Appointment.java`
- `EmergencyCase.java`

10.2 Service Package

`HospitalService.java` contains operations for managing hospital records and performing operations involving patients, doctors, appointments, and emergency cases.

Collections such as `ArrayList` are used to maintain multiple records.

10.3 Exception Package

`AppointmentException.java` is a user-defined exception created for handling invalid appointment-related situations.

This allows the program to provide controlled error handling instead of allowing invalid operations to continue.

10.4 Object-Oriented Implementation

The project uses inheritance to create specialized doctors from the common `Doctor` class.

For example:

```
None
Doctor
  ├── Cardiologist
  ├── Dentist
  ├── Neurologist
  └── OrthopedicDoctor
```

Method overriding is used to provide specialist-specific behavior, demonstrating polymorphism.

11. Design Decisions and Rationale

The project was designed around a modular structure because hospital management contains several different types of information.

Separate classes were used for patients, doctors, appointments, and emergency cases so that each entity has its own responsibility.

Inheritance was selected for specialist doctors because different specialists share common doctor-related properties while still having their own behavior.

`ArrayList` was selected for storing records because the number of patients, doctors, and appointments can change during execution.

A custom exception was used for appointment validation so that invalid operations can be handled clearly.

The web interface uses a dark glassmorphism and 3D-inspired design to provide a modern presentation while keeping the major hospital modules separated.

12. Results / Output

The completed project provides a centralized interface containing multiple hospital-management modules.

Users can navigate through patient management, doctor management, appointments, emergency command, hospital operations, and analytics.

The Doctor Management section provides multiple specialists and their information. The Emergency Command section separates emergency information into priority queue, active cases, available emergency doctors, and emergency history.

The Java module successfully demonstrates the selected Programming in Java concepts using hospital-related examples.

The Java source can be compiled and executed separately using a standard Java Development Kit.

13. Testing Approach

Testing was carried out at both the application and Java levels.

Web Application Testing

The following areas were checked:

- Landing page navigation
- Dashboard navigation
- Patient management
- Doctor management
- Appointment functionality
- Emergency sections
- Hospital operations

- Analytics
- Navigation drawer
- Interactive interface elements

Java Testing

The Java module was tested by compiling and executing the source files.

Testing includes:

- Creating patient objects
- Creating doctor objects
- Creating specialist doctor objects
- Testing inheritance
- Testing method overriding
- Testing polymorphism
- Testing getters and setters
- Adding records to `ArrayList`
- Searching and processing records
- Creating appointments
- Testing invalid appointment conditions
- Catching custom exceptions

Example compilation:

None

```
javac -d bin src/model/*.java src/exception/*.java  
src/service/*.java src/Main.java
```

Example execution:

None

```
java -cp bin Main
```

14. Challenges Faced

During development, some of the main challenges were:

- Organizing multiple hospital modules into one application.
- Maintaining consistent navigation between modules.
- Managing different doctor specializations.
- Handling invalid appointment operations.
- Organizing Java classes into appropriate packages.
- Connecting the practical hospital scenario with Java programming concepts.
- Maintaining a modern interface without making the application difficult to navigate.

These challenges helped in understanding how programming concepts can be combined to build a larger application rather than using them as isolated examples.

15. Learnings and Key Takeaways

This project provided practical experience with Java programming and software development.

The major learning outcomes include:

- Understanding how classes and objects represent real-world entities.
- Applying encapsulation to protect object data.
- Using inheritance to create specialized doctor classes.
- Understanding runtime polymorphism through method overriding.
- Handling invalid operations using exceptions.
- Creating and using user-defined packages.
- Using `ArrayList` for dynamic data management.
- Applying loops and conditional statements to process records.
- Understanding how different modules can work together in a larger application.
- Improving the organization and maintainability of source code.

The project also provided experience in designing a user interface for a practical software application.

16. Future Enhancements

MediSphere 3D can be expanded further by adding:

Database Integration

A MySQL or PostgreSQL database could be used to permanently store patients, doctors, appointments, and emergency records.

Authentication

Different roles could be introduced for:

- Administrators
- Doctors
- Nurses
- Reception staff
- Patients

Real-Time Monitoring

Hospital equipment and operational systems could be integrated to provide real-time information.

Notifications

Email, SMS, or application notifications could be added for appointments and important hospital updates.

Advanced Analytics

More detailed analytics could help with hospital capacity, appointment demand, and resource planning.

Mobile Application

A dedicated Android or iOS application could make hospital services accessible through mobile devices.

17. Conclusion

MediSphere 3D demonstrates the application of Programming in Java concepts to a practical hospital-management problem.

The project brings together patient management, doctor management, appointment management, emergency management, hospital operations, and analytics in one system.

The Java implementation provides practical use of classes and objects, encapsulation, inheritance, polymorphism, exception handling, user-defined packages, collections, loops, and other core Java features.

Rather than treating these concepts as separate examples, the project uses hospital-related entities to show how they can work together in a larger application.

The current system provides a foundation that can be extended with database connectivity, authentication, real-time monitoring, and other advanced features in the future.

18. References

1. Oracle Java Documentation
2. Java Platform Documentation
3. MDN Web Docs
4. React Documentation
5. Vite Documentation
6. GitHub Documentation

19. Screenshots

MediSphere 3D

SMART HOSPITAL MANAGEMENT SYS...

Home

Search patients, doctors, appointments, triage, departments...

Status: 99.4% Operational

11:38 PM

MediSphere 3D

SMART HOSPITAL MANAGEMENT SYS...

PORTAL NAVIGATION - 7 MODULES

Dashboard

Patient Management

Doctor Management

Appointments

Emergency Command

Hospital Operations

Analytics & System

VTOP HOSPITAL OS - MODULE 7

Intelligence & 3D Analytics

Analytics & System Intelligence

Real-time hospital KPI dashboards, structured clinical reporting, and live Java concurrency monitoring with active worker threads/pools and database synchronization.

Live Telemetry

Export Data

3 Running

Audit Logs & DB

Hospital Analytics

Occupancy Rates, Clinical KPI Charts & Revenue...

Reports

Automated CSV, Census Rosters & Discharge...

System Monitor

System Performance, Background Tasks &...

System Activity

Connection Pool, Diagnostics & System Config

VIT BHOPAL - PROGRAMMING IN JAVA

Compileable Java Module (JVM/src)

Java Implementation

Genuine Java SE codebase implementing the six syllabus concepts for hospital clinical workflows. Real .java source files are organized in model, exception, and service packages inside the repository.

SIX MAJOR JAVA CONCEPTS IN MEDISPHERE

model.Patient / doctor / Appointment

private fields + defensive setters

extends doctor + super(...)

CLASSES & OBJECTS

Patient, Doctor and Appointment are represented as Java classes.

Instantiated using constructors with real hospital parameters (e.g. patientId, bloodGroup, specialization, consultationFee).

ENCAPSULATION

Patient and doctor information is protected using private fields.

Getters and setters enforce validation: age cannot be negative, name cannot be empty, consultation fee cannot be negative.

INHERITANCE

Specialized doctors inherit common properties from Doctor, Cardiologist, Dentist, Neurologist, and OrthopedicDoctor extend Doctor using "extends Doctor" and constructor chaining with super(...).

DYNAMIC METHOD OVERRIDING

AppointmentException + try-catch

AppointmentException + try-catch

POLYMORPHISM

Different specialists override consultationDetails().

Exception Handling

Invalid appointment operations are handled using custom

ArrayList Collections

Hospital records are managed using ArrayList.

Terminal Compilation

Copy

javac -d bin src/model/* -java src/exception/* -java src/service/* -java src/H

MediSphere 3D

SMART HOSPITAL MANAGEMENT SYSTEM

Explore System

ENTERPRISE CLINICAL CAPABILITIES

Powerful Features for a Smarter Healthcare

Engineered for high-reliability medical campuses. Every subsystem seamlessly interconnects patient telemetry, specialist coordination, and real-time operations.

Hospital Core

Smart Hospital Operations

Integrated digital platform orchestrating admissions, bed allocations, and clinical workflows across all wings.

Real-time Triage

Emergency Management

Algorithmic real-time triage prioritizing Critical, High, Medium, and Low cases with automated physician dispatch.

Smart Scheduling

Appointment Intelligence

Zero-conflict multi-specialty scheduling across 25 medical domains with automated conflict detection and timeline visualization.

High Throughput

Laboratory & Pharmacy

Automated specimen processing pipelines paired with robotic/formulary inventory tracking and low-stock alerts.

HOLOGRAPHIC PATIENT TELEMETRY

SCAN ACTIVE + 99.4% INTERACTIVITY

75 bpm

Normal Range

99.8% NORMAL

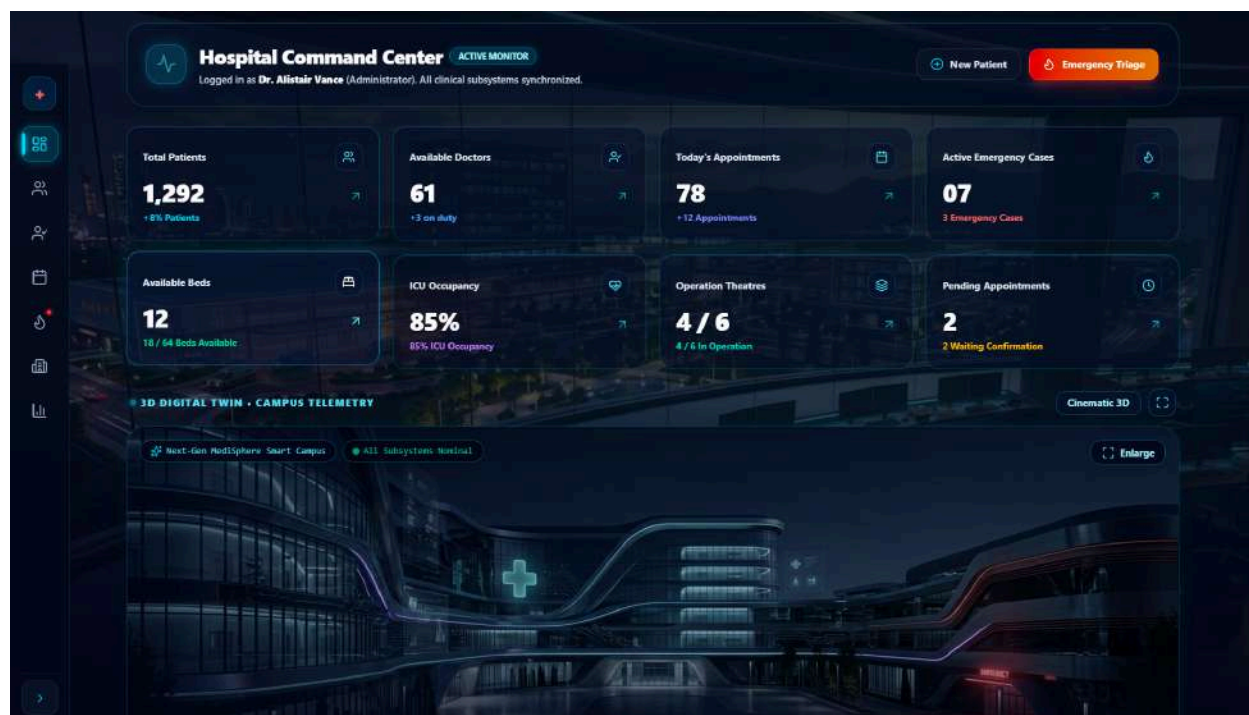
Alpha Spectroscopy

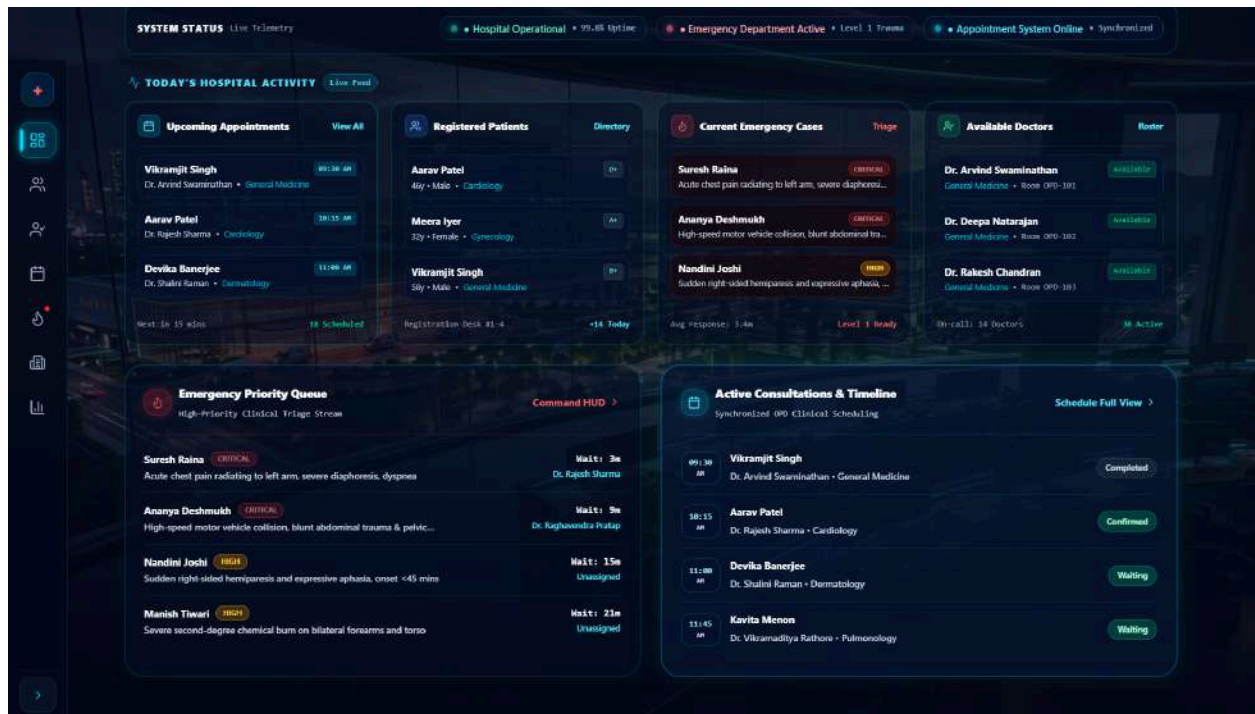
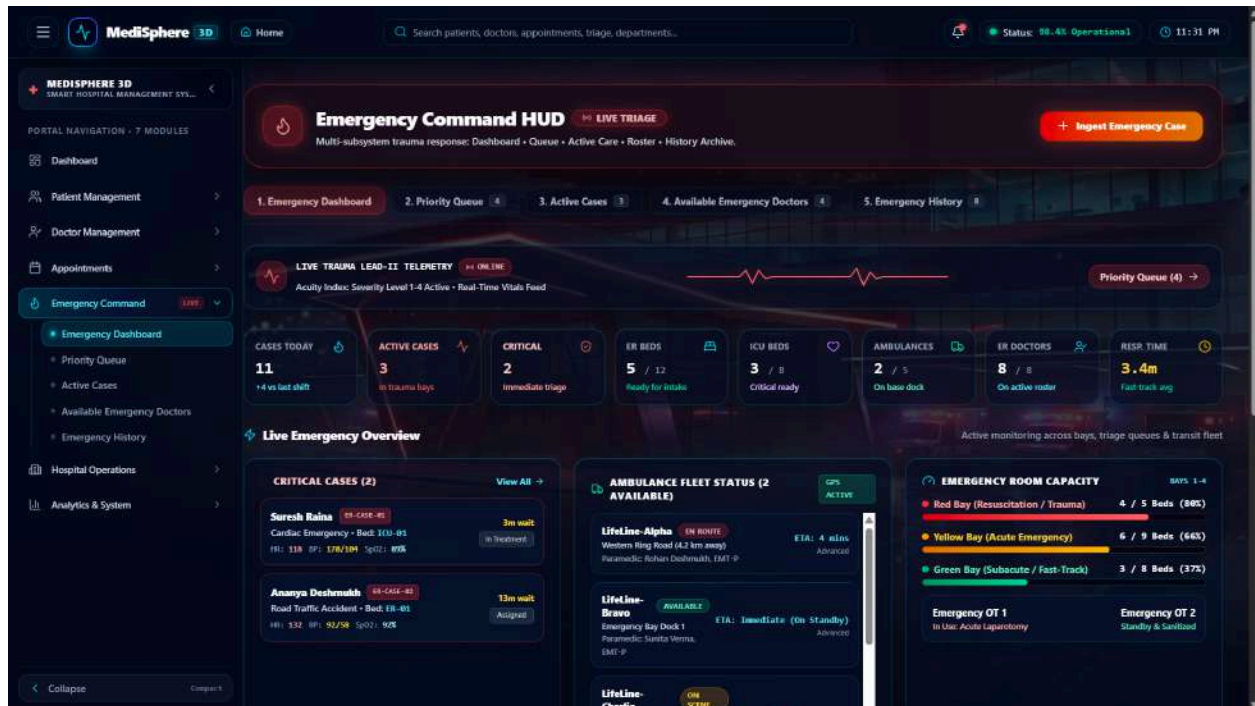
98% SpO2

O2 Saturation

120/80 mmHg

Normal





MediSphere 3D

Home

Search patients, doctors, appointments, triage, departments...

Status: 98.4% Operational11:11 PM

MediSphere 3D

SMART HOSPITAL MANAGEMENT SYS...

PORTAL NAVIGATION - 7 MODULES

Dashboard

Patient Management

Doctor Management

Appointments

Emergency Command

Hospital Operations

Laboratory

Pharmacy

Bed Management

Billing & Invoices

Thread Activity Monitor

Analytics & System

VTOP HOSPITAL 05 - MODULE 6

Care Operations

Hospital Operations Command

Unified clinical operations center integrating automated diagnostic pathology, pharmacy stock dispensing, live bed occupancy grid, and GST revenue invoicing.

8 Active Assays

Laboratory

Hematology, Bio-Chemistry &...

6 Alerts

Pharmacy

Formulary Inventory & Barcoded...

12 Available

Bed Management

ICU, General Ward & Critical Care...

2 Pending

Billing & Invoices

GST Invoicing, Claims & Ledger...

5 Active

Thread Activity Monitor

Java Multithreaded Operations &...

Java Multithreaded Operations Engine

5 / 7 Workers Active

8 Backlogs

5 Reentrants/locks Synchronized

Background tasks concurrently servicing diagnostic pathology, pharmacy CAD dispensing, ward miscues, and billing commits.

Quick Process Panel

Full Process Charts

Diagnostic Pathology & Imaging

8 In Queue

Multithreaded automated analyzers, radiological modalities, and reference range validation.

Run Batch Analysis

Order Diagnostic Test

Search test name, patient ID, doctor...

Queue State: All States

LAB-700

COMPLETED

Lipid Profile

Patient: Aarav Patel

Specimen:

Requested By:

Diagnostic Fee:

VERIFIED PARAMETERS

LAB-702

COMPLETED

ECG

Patient: Aarav Patel

Specimen:

Requested By:

Diagnostic Fee:

VERIFIED PARAMETERS

LAB-703

PROCESSING

CBC

Patient: Meera Iyer

Specimen:

Requested By:

Diagnostic Fee:

VERIFIED PARAMETERS